Republic of the Philippines
CAMARINES SUR POLYTECHNIC COLLEGES
Nabua, Camarines Sur

COLLEGE *of* COMPUTER STUDIES

A Computer Vision Project Report

MIDTERM PROJECT REPORT

Submitted as a requirement for the course
Computer Vision

Group Project Specialization

Delivery Robot

Prepared by:

Aguilar, Vince Clifford C.
Benosa, Nhel Adam S.
Caldo, John Paul B.
Llesis, Roinel James A.
Olivares, Elyssa
Sario, Joshua Q.

BS Computer Science – 4B

Submitted to:

Elena Clarexe Gulane
Course Instructor

1. INTRODUCTION

1.1 Project Overview

This project develops the vision system of a four-wheel-drive (4WD) autonomous delivery robot. The robot simulates a delivery vehicle that must carry a package to the correct destination, and that destination is identified visually rather than through a fixed route or a hard-coded location. In the full mission, the robot starts from a designated starting point and navigates through the course. It monitors its surroundings with a camera, detects the available destination markers, and identifies which one matches its assigned destination. It then drives to the corresponding delivery zone, stops inside it, and signals a successful delivery with an LED or buzzer.

Vision is the first and most critical link in this chain. The robot can only deliver to the right place if it can first tell which place it is looking at. This project therefore treats the camera as the robot's primary sensor and uses machine learning to recognize printed destination markers. Five markers represent the five possible destinations: A (square), B (triangle), C (circle), D (star), and E (arrow). Each is a printed symbol on a post-like sign placed at its delivery zone.

The system follows a SEE → RECOGNIZE / INTERPRET → DECIDE → ACT pipeline. This Midterm report covers the first stage, SEE → RECOGNIZE. Given an image from a camera, the system recognizes which of the five markers is present and reports a confidence score for its prediction, so that uncertain images can be rejected instead of trusted. The recognition model is a MobileNetV2 convolutional neural network, pretrained on ImageNet and fine-tuned on frames extracted from videos of the printed markers. It was chosen because it is small and fast enough to run later on the limited computing hardware of a mobile robot.

The model was trained and evaluated on **3,756 images** in total: 2,000 frames captured under training conditions, and 1,756 frames captured in dim lighting against a different background, which serve as an unseen test set. A live webcam demonstration shows the predicted label and confidence in real time.

1.2 Problem Description

The primary problem addressed by this specialization is destination recognition and delivery verification, not searching for an arbitrary object. The robot is assigned one destination, for example B. As it moves through the course it will pass markers belonging to other destinations, such as A or C, and it must continue past them. It should stop only when it recognizes its own assigned marker, which confirms that it has arrived at the correct place. A wrong recognition has real consequences: if the robot mistakes marker A for marker B, it delivers the package to the

wrong destination. Reliable discrimination between all five markers is therefore the core requirement.

This is difficult because the robot will not see the markers under ideal or consistent conditions. As it moves, the same marker will appear at different distances, from different viewing angles, and with different rotations and tilts. The background will vary from place to place along the course, and the lighting may range from normal to dim. A classifier that only works on one clean, well-lit view of each marker would fail in practice. The model must also perform well on conditions it was never trained on, since the robot cannot be assumed to operate in the exact setting where the training data was recorded.

A further problem is that the robot will often have no marker in view, such as when it is driving along an empty stretch of the course, facing a wall, or looking at the floor. A standard classifier always outputs one of its known classes, even for an image that contains no marker, and may do so with misleading confidence. The system must therefore also decide when it is unsure and avoid acting on those images. For this reason the classifier reports a confidence score, and predictions below a set threshold (70%) are labeled “unknown” and rejected.

The problem addressed in this Midterm can therefore be stated as follows: to build and evaluate an image classifier that recognizes the five destination markers (A, B, C, D, and E) under varying distance, angle, rotation, background, and lighting. The classifier should achieve high accuracy on both same-condition test data and a separate unseen-condition set, and should flag low-confidence inputs instead of acting on them. It must also be lightweight enough to be deployed later on the delivery robot.

1.3 Project Objectives

General Objective

To develop and evaluate a computer vision system for a 4WD delivery robot that recognizes the five destination markers (A, B, C, D, and E) from camera images and reports a confidence score, so that the robot can later use this recognition to identify its assigned delivery destination.

Specific Objectives

To construct, evaluate, and deploy a transfer-learning vision model for printed marker identification, this study specifically sought to**:**

1. To build a dataset of the five printed markers by recording videos at varying distances, tilts, and rotations, extracting frames with OpenCV, and dividing them by time segment into training, validation, and testing sets, plus a separate unseen set recorded in dim lighting with a different background.
2. To train an image classifier using transfer learning (ImageNet-pretrained MobileNetV2 with data augmentation) that recognizes all five markers.
3. To achieve at least 95% accuracy on the held-out test set and at least 90% accuracy on the unseen dim-light set, evaluated with per-class precision, recall, F1-score, and confusion matrices.
4. To apply a 70% confidence threshold so that low-confidence predictions are labeled “unknown” instead of being accepted as a marker.
5. To demonstrate the saved model on a live webcam feed that displays the predicted label and confidence in real time.

1.4 Scope and Limitations

This study focuses on the development of a destination recognition system for Autonomous Delivery Robot. The primary operational scope involves the image classification of five distinct visual classes: labeled as A (Square), B (Triangle), C (Circle), D (Star), and E (Arrow). Each target destination consists of a post-like signage with a printed image of the target class/shape with a measured height with the camera angle. The mid-level classification pipeline operates in indoor environments, processing frames captured from a laptop webcam or mobile phone camera at distances ranging from approximately 0.3 m to 2 m under varying pitch, tilt, rotation, normal and dim lighting conditions, and plain indoor backgrounds. In terms of processing, each incoming frame undergoes color conversion from BGR to RGB and is resized to 224×224 pixels before being fed into an ImageNet-pretrained, fine-tuned MobileNetV2 convolutional neural network. The network outputs Softmax probabilities, and a threshold confidence check 70% determines whether the frame is assigned a high-confidence class label or designated as "unknown" for on-screen display.

Conversely, the system's current implementation is subject to several key limitations. The pipeline performs image-level visual classification returning a single label and confidence score per frame without localized bounding boxes or spatial detection of where the marker appears within the image frame. Furthermore, the classifier does not incorporate a dedicated "no marker" training class, relying solely on the proposed 70% confidence threshold to filter out non-target inputs. Hardware and dataset diversity are also constrained; training and validation were conducted using only a single set of printed markers and one mobile capture device, meaning the model has yet to be evaluated on the physical delivery robot's embedded camera.

1.5 Group Project Specialization

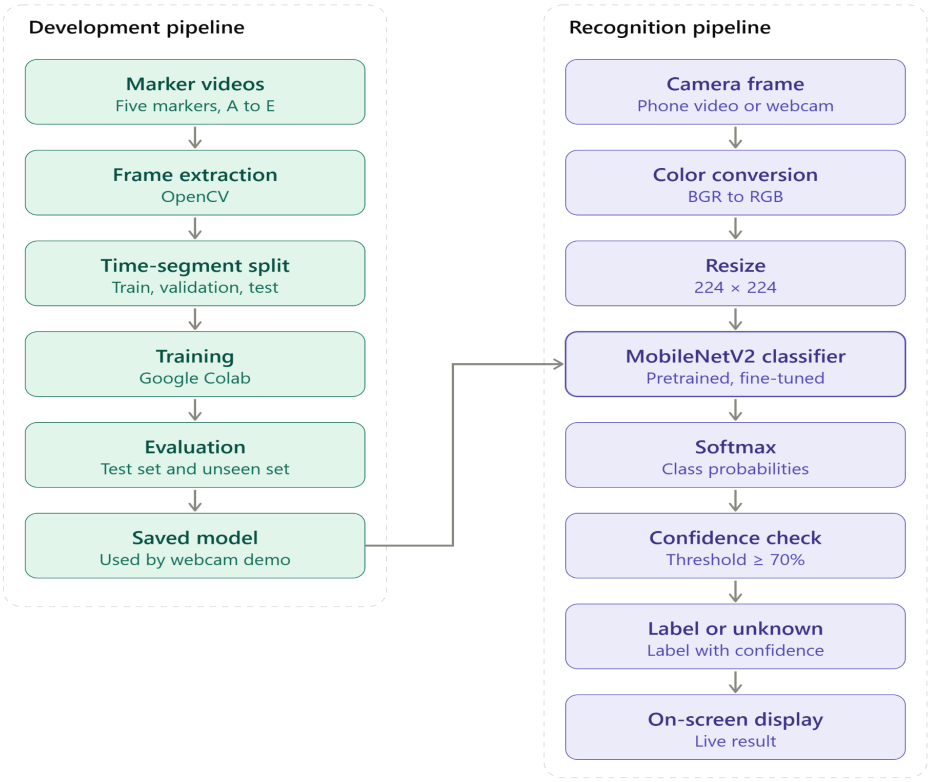
The group's specialization is the Delivery Robot, and the required computer vision task is to recognize destination markers so that the robot can identify the location where it must deliver a package. In the complete specialization, the robot is assigned one of five destinations (A, B, C, D, or E), navigates the course, identifies the matching marker, stops inside the corresponding delivery zone, and signals a successful delivery with an LED or buzzer.

This Midterm report covers the SEE → RECOGNIZE stage of that specialization: recognizing which of the five markers appears in a camera image and reporting a confidence score. The remaining stages, which are deciding whether the recognized marker matches the assigned destination, navigating to the delivery zone, stopping, and signaling delivery, are reserved for the Final Project.

2. SYSTEM DESIGN AND ARCHITECTURE

2.1 Overall System Architecture

Present a diagram showing the major components of the system and how they interact.



2.2 Computer Vision Workflow

1. Print five markers on A4 paper and record one video per marker while moving the camera closer and farther, tilting, and rotating the paper.

2. Extract frames from each video using OpenCV.

3. Split frames by time segment into training, validation, and testing sets.

4. Record a second set of videos in dim lighting and a different background, and extract frames as the unseen test set.

5. Train MobileNetV2 in two stages (new classification head, then fine-tuning of the top layers) with data augmentation.

6. Evaluate on the held-out test set and on the unseen set, using accuracy, precision, recall, F1-score, and confusion matrices.

7. Run the saved model on a live webcam feed and display the label and confidence.

2.3 Hardware Components

A table is recommended:

Component	Specification / Model	Purpose
Camera (dataset capture)	Smartphone camera, 1920 × 1080 video using a phone	Recording the marker videos
Camera (live demo)	Laptop built-in webcam	Real-time recognition demo
Computer	Laptop running Windows [laptop model]	Frame extraction, dataset preparation, demo
Training hardware	Google Colab GPU runtime (T4)	Model training and evaluation
Markers	Printed A4 paper, five symbols (sample for the model training)	Destination markers

2.4 Software and Development Tools

Identify the programming languages, libraries, frameworks, platforms, and development tools used.

Software / Tool	Version, if applicable	Purpose
NumPy, Matplotlib	Colab default	Array handling, plots, figures
scikit-learn	Colab default	Classification report and confusion matrix
TensorFlow / Keras	Colab default [version]	Model building and training
Python	3.13 (local); Colab default (training)	Programming language
Google Colab, Google Drive		Training environment and file storage
OpenCV (opencv-python)	5.0.0	Video reading, frame extraction, webcam capture, on-screen display

3. DATASET

3.1 Dataset Description

The dataset consists of images of five marker classes (A, B, C, D, and E) used to train an image classification model for the Delivery Robot. It has two parts. The main dataset is split into training, validation, and testing sets captured under the same conditions. A separate unseen test set (raw_test) was captured under dim lighting with a different background to check how well the model handles conditions it was not trained on. In total, 3,696 images were used.

3.2 Target Classes / Categories

The five classes are balanced. Each class makes up between 19% and 22% of the training-condition frames (A 19.8%, B 19.3%, C 20.0%, D 22.1%, E 19.0%), so no class weighting or resampling was needed. The unseen set is similarly balanced (317 to 378 frames per class).

3.3 Dataset Collection and Source

Class	Description	Training-condition frames	Unseen (dim) frames
A	Solid magenta square with a black border on white paper	395	336
B	Triangle with a black border on white paper	385	359
C	Circle symbol on white paper	399	378
D	Star with a black border on white paper	441	366
E	Arrow with a black border on white paper	380	317
Total		**2,000**	**1,756**

The dataset was collected entirely by group member Nhel Adam Benosa using a smartphone camera. For each marker, one video was recorded while slowly moving the camera closer and farther, tilting it, and rotating the paper. Frames were extracted with OpenCV, saving every 10th frame. The unseen set was recorded afterwards for each marker, in dim lighting and in a different setting from the original videos, and frames were extracted by saving every 5th frame because these videos were shorter. No external datasets were used.

3.4 Dataset Distribution

Figure 1 shows the share of each marker class among the 2,000 frames captured under training conditions. The five classes are well balanced, each making up between 19.0% and 22.1% of the dataset: A (Square) with 19.8%, B (Triangle) with 19.3%, C (Circle) with 20.0%, D (Star) with 22.1%, and E (Arrow) with 19.0%. D (Star) is the largest class with 441 frames, and E (Arrow) is the smallest with 380 frames, a difference of only about 3 percentage points. Because no class dominates the dataset, the model is not biased toward any one marker during training, so class weighting and resampling were not needed. Percentages may not sum to exactly 100% due to rounding.

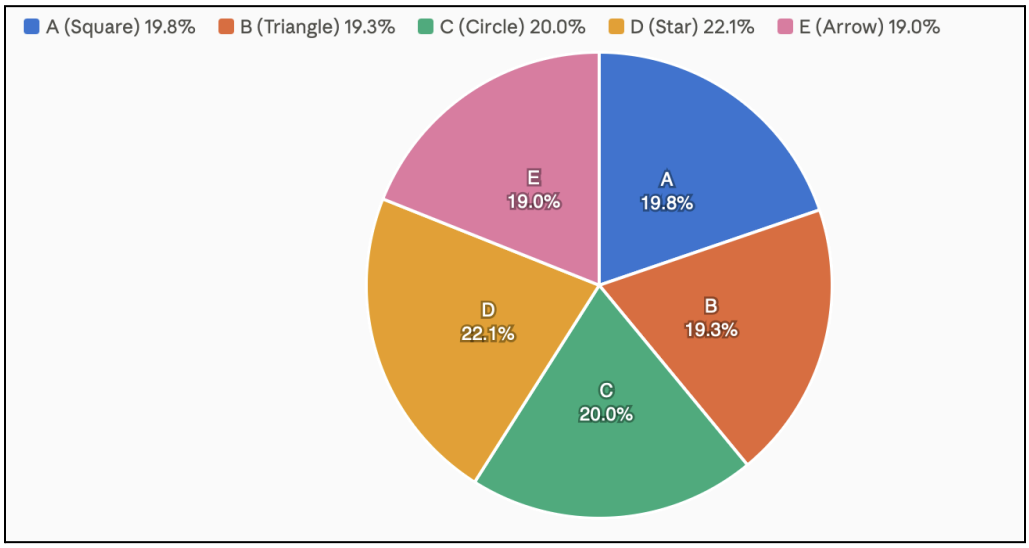


Figure 1. Class Distribution of the Training-Condition Frames (n = 2,000)

3.5 Training, Validation, and Testing Sets

Document on how the dataset was divided.

Dataset	Number of Samples	Percentage
Training	1,383	71.3%
Validation	270	13.9%

Testing (same conditions)	287	14.8%
Total	**1,940**	**100%**
Unseen test set (dim light, new background)	1,756	evaluated separately

Per-class counts:

Class	Train	Validation	Test	Unseen
A	273	53	57	336
B	266	52	55	359
C	276	54	57	378
D	305	60	64	366
E	263	51	54	317

The split was done by time segment within each video: the first 70% of frames for training, the next 15% for validation, and the last 15% for testing. A gap of 3 frames was skipped at each boundary, which accounts for the 60 frames (2,000 − 1,940) not used. This prevents near-identical neighboring frames from appearing in different sets.

- **Training set:** used to fit the model weights.
- **Validation set:** used to monitor performance during training and detect overfitting.
- **Testing set:** used only for final evaluation under conditions similar to training.
- **Unseen set:** used only for final evaluation under new lighting and background, representing situations not seen in training.

3.6 Dataset Preparation and Preprocessing

Frames were extracted with OpenCV and saved as JPEG files. All extracted frames were kept; none were cropped or removed. Images were resized to 224 × 224 pixels by the data loader when training and testing. The split was done by time segment (see 3.5) rather than at random, and the unseen set was kept entirely separate from training and validation.

4. COMPUTER VISION METHODOLOGY

4.1 Computer Vision Approach

The group used image classification with transfer learning. A convolutional neural network pretrained on ImageNet already detects edges, shapes, and textures, so it can be adapted to the five markers with a small dataset and short training time. This suits the project because the markers are visually distinct, the dataset is small (under 2,000 images), and the model must later run on a robot with limited computing power. MobileNetV2 was selected because it is small and fast.

4.2 Image Processing / Preprocessing

Step	Setting
Resize	224 × 224 pixels (stretched, aspect ratio not preserved)
Pixel scaling	MobileNetV2 preprocess_input (scales values to −1 to 1), built into the model
Rotation (training only)	Random, factor 0.08
Zoom (training only)	Random, factor 0.2
Translation (training only)	Random, 10% horizontally and vertically
Brightness (training only)	Random, factor 0.5 on a 0–255 range
Contrast (training only)	Random, factor 0.4

Horizontal flipping was intentionally not used, because a mirrored marker could resemble a different class. Wide brightness variation was included so the model sees dark images during training.

4.3 Model / Algorithm

The model is MobileNetV2 (Sandler et al., 2018) with ImageNet weights, a lightweight convolutional neural network designed for mobile devices. Its original 1,000-class classifier was removed and replaced with a new head: global average pooling, dropout (0.3), and a dense layer with 5 outputs and softmax activation. The softmax output provides the confidence score for each class.

4.4 Model Training / Development

Training had two stages in Google Colab:

Setting	Stage 1: train new head	Stage 2: fine-tuning
MobileNetV2 body	Frozen	Top 30 layers unfrozen
Epochs	10	8
Optimizer	Adam	Adam
Learning rate	1e-3	1e-5
Loss function	Sparse categorical cross-entropy	Sparse categorical cross-entropy
Batch size	32	32

4.5 Model Parameters and Configuration

Parameter	Value
Input size	224 × 224 × 3
Number of classes	5
Base model	MobileNetV2, ImageNet weights
Dropout	0.3

Total epochs	18 (10 + 8)
Batch size	32
Data-loading seed	42
Confidence threshold (demo)	0.70
Saved model format	marker_model.keras with class_names.json

4.6 Detection / Recognition Process

Camera Frame → Convert BGR to RGB → Resize to 224 × 224 → Model (preprocessing, MobileNetV2, classification head) → Softmax probabilities → Highest-probability class → If confidence ≥ 70%, show the class label; otherwise show 'unknown'.

5. SYSTEM IMPLEMENTATION

5.1 Computer Vision Implementation

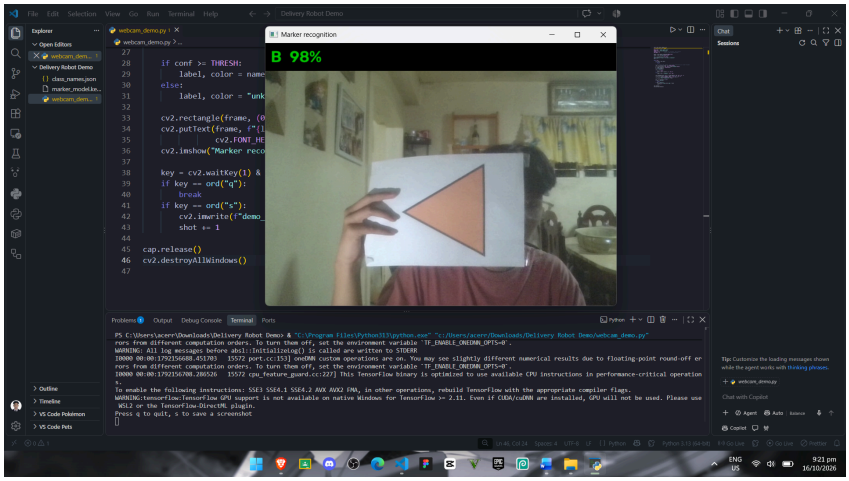
The system was implemented with four components: a Python script that extracts frames from the marker videos with OpenCV, a script that performs the time-segment split, a Google Colab notebook that trains and evaluates the model, and a Python webcam script that loads the saved model and performs live recognition. Training used the Colab GPU, and the saved model was run on the laptop for the demo.

5.2 Input and Camera Processing

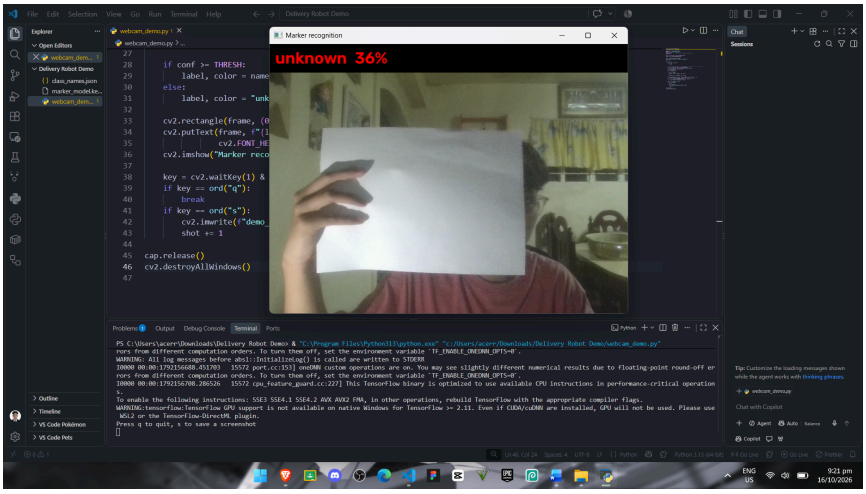
For dataset creation, videos were read with cv2.VideoCapture and every 10th frame (every 5th for the unseen videos) was saved as an image. For the live demo, frames are read from the laptop webcam with cv2.VideoCapture, converted from BGR to RGB, resized to 224 × 224, and passed to the model one frame at a time.

5.3 Visual Output

The demo overlays a black banner at the top of the video window showing the predicted label and its confidence percentage, for example 'C 98%'. The text is green when confidence is at least 70% and red, reading 'unknown', when it is below. Pressing 's' saves a screenshot and 'q' quits.



Screenshot of Recognized Mark



Screenshot of Unrecognized Mark

6. TESTING AND EVALUATION

6.1 Computer Vision Testing

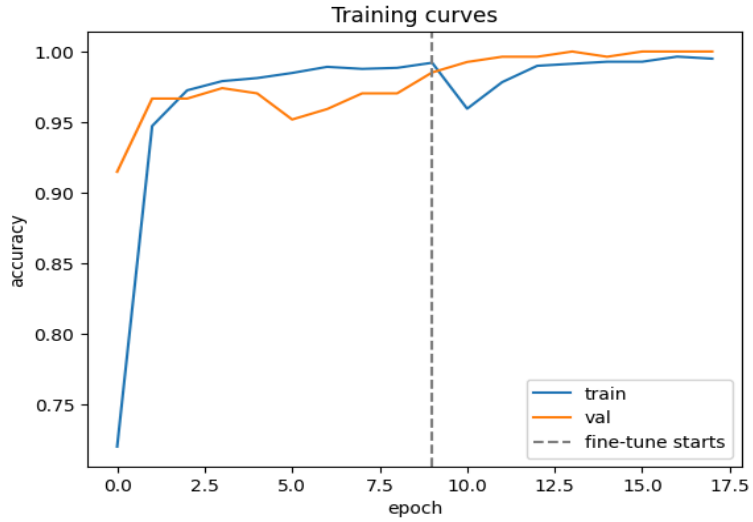
The system was tested in three ways. First, the validation set was evaluated after every training epoch. Second, the held-out test set (287 frames, last time segment of each video) measured performance under conditions similar to training. Third, the unseen set (1,756 frames, dim lighting and a different background) measured performance in a situation not used for training or validation. Finally, the live webcam demo was tried with printed markers and with a blank white paper

6.2 Evaluation Metrics

Accuracy, precision, recall, F1-score, and confusion matrices were used. These suit a balanced multi-class classification task

6.3 Training, Validation, and Testing Results

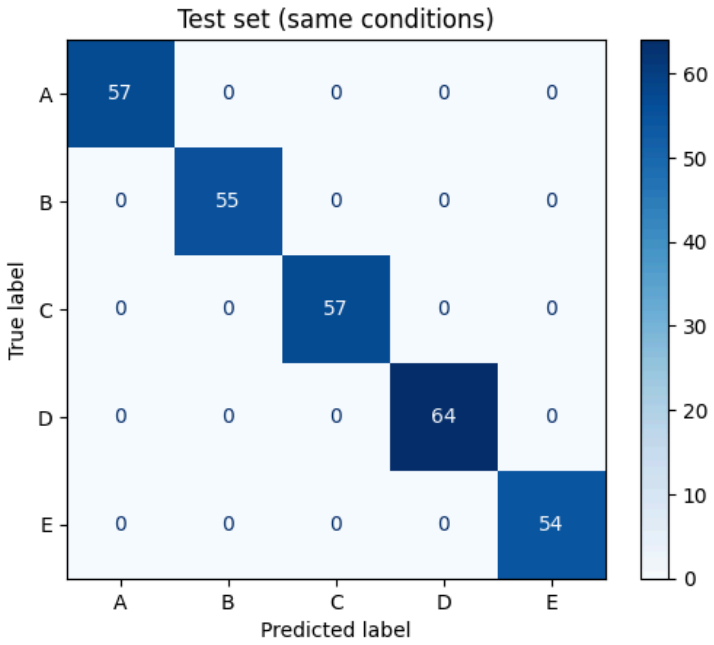
During training, accuracy rose from about 72% (training) and 91% (validation) in the first epoch to about 99.5% (training) and 100% (validation) by the end. Validation accuracy is often higher than training accuracy because augmentation is applied only during training, which makes the training task harder. A small dip occurred when fine-tuning started, which is expected after unfreezing layers.



Confusion Matrix for Training curves

Test set (same conditions), accuracy 100% (287 frames):

Class	Precision	Recall	F1-score	Support
A	1.000	1.000	1.000	57
B	1.000	1.000	1.000	55
C	1.000	1.000	1.000	57
D	1.000	1.000	1.000	64
E	1.000	1.000	1.000	54



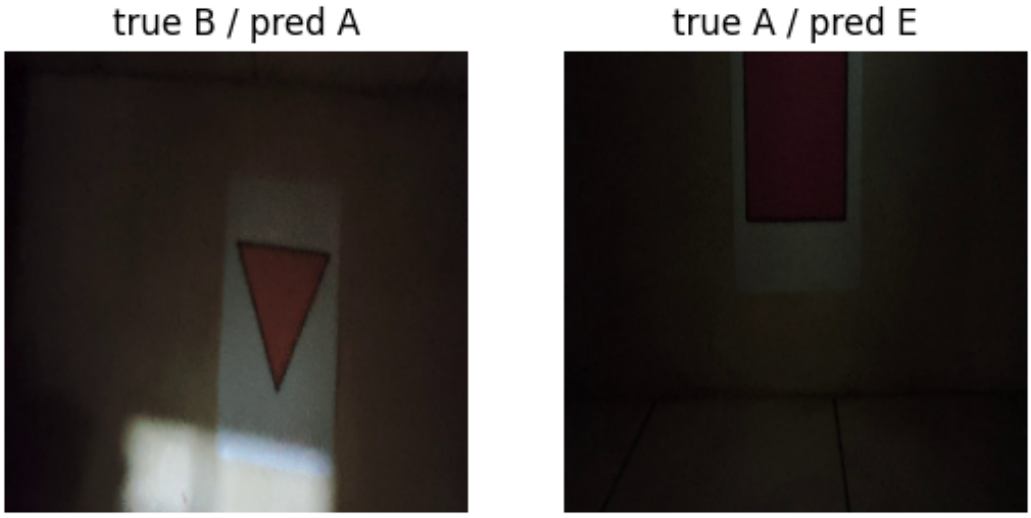
Confusion Matrix for Test Set

6.4 Sample Detection / Recognition Results

Successful results:

- In the live webcam demo, the printed circle marker was recognized as class C with 98% confidence.
- A blank white paper held in front of the camera produced 'unknown', showing that the confidence threshold rejects images that contain no marker

Unsuccessful results: On the unseen set, 2 of 1,756 frames were misclassified: one frame of marker A was predicted as E, and one frame of marker B was predicted as A. These were likely frames affected by heavy darkness, noise, blur, or an extreme viewing angle. See the figure below

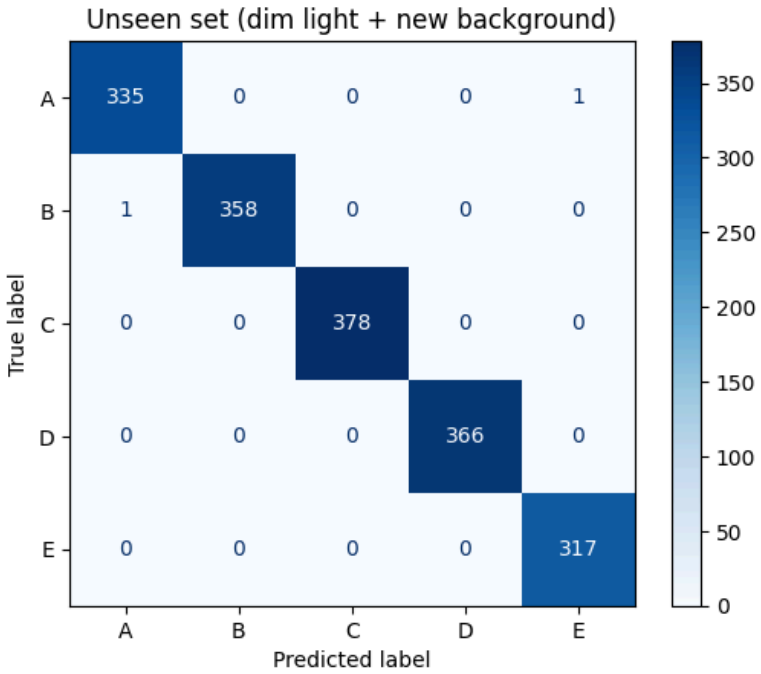


Misclassified frames from the unseen set

6.5 Unseen Situations

The unseen set used dim lighting and a different background from the training videos. The model was trained with strong brightness augmentation but never saw these recordings.

Evaluation set	frames	accuracy
Test set (same conditions)	287	100.00%
Unseen set (dim light, new background)	1,756	99.89%



Confusion matrix, unseen set

Performance dropped by only about 0.1 percentage points. The likely reasons are the strong visual differences between the markers, the high contrast of the printed markers, and the brightness and contrast augmentation used in training. This result should be read with care: the unseen frames come from five videos, so neighboring frames are very similar, and the same printed markers and phone were used.

7. RESULTS AND DISCUSSION

7.1 Computer Vision Results

The MobileNetV2 classifier recognized all five markers with 100% accuracy on the held out test set and 99.89% on the unseen dim-light set. The model therefore met all the objectives set for the Midterm. The live demo confirmed that the model runs in real time on a laptop webcam.

7.2 Analysis of Results

What worked well: transfer learning gave very high accuracy with fewer than 1,400 training images; the time-segment split gave a fair test; brightness augmentation helped the model handle dim lighting; the confidence threshold rejected a blank paper.

What did not work well: two unseen-set frames were misclassified, and the model has no way to say 'no marker' other than low confidence.

Patterns and factors: the few errors occurred only in the dim set. Lighting, image noise, viewing angle, and marker distance are the main factors that affect performance. Because the markers differ clearly in shape, the task is relatively easy, which partly explains the near-perfect scores.

7.3 Errors and Failure Cases

The two errors were one A frame predicted as E and one B frame predicted as A in the unseen set. Likely causes are low light with image noise, blur, or an unusual viewing angle that made the marker harder to read. These errors are minor on a 1,756-frame set, but on a robot a single wrong label could send it to the wrong destination, so a later system should average predictions over several frames before acting.

8. PROBLEMS ENCOUNTERED AND LIMITATIONS

8.1 Problem encountered:

- The first recording was done in a single lighting condition only, so there was no unseen lighting test. The group recorded a second set in dim light to fix this.
- Neighboring video frames are nearly identical, so a random split would have inflated test accuracy. A time-segment split with boundary gaps was used instead.
- Installing OpenCV in Python required the package name opencv-python (the name cv2 is not a valid package)

8.2 Limitations:

- The system classifies a whole frame and does not locate the marker, so it cannot tell the robot whether the marker is left, right, near, or far.
- There is no 'no marker' class. Rejection of empty scenes relies only on the 70% confidence threshold, and only a blank white paper was tried; floors, walls, people, and cluttered backgrounds were not systematically tested.
- Partial occlusion, long distances, and motion blur were not tested. Only one printed set of markers, one phone, and five videos were used, so neighboring frames are highly similar.
- The model has not been tested on the robot's own camera or on the robot's computer.

9. CONCLUSION AND RECOMMENDATIONS

9.1 Conclusion

[The group built a working computer vision system that recognizes five destination markers using a fine-tuned MobileNetV2. The dataset of 2,000 frames was split by time segment into training, validation, and test sets, and a separate 1,756-frame unseen set in dim light and a new background was used to test unseen situations. The model reached 100% accuracy on the test set and 99.89% on the unseen set, and ran live on a webcam with label and confidence output. The Midterm requirement, a system that can see and recognize, was achieved.] - *initial conclusion for midterm completion*

9.2 Recommendations

1. Add a 'no marker' class with images of floors, walls, people, and cluttered scenes, and
2. retrain, so the robot does not act on empty views.
3. Add marker localization (for example bounding-box detection or contour detection
4. with this classifier) so the robot can steer toward the marker.
5. Record additional training and test data with the robot's own camera, at its mounting
6. height, and while it is moving, including motion blur.
7. Test occlusion, long distances, strong angles, and more varied lighting and
8. backgrounds, and report them separately.
9. Require the same prediction in several consecutive frames before the robot acts.
10. Convert the model to TensorFlow Lite if the robot's computer is slower than a laptop.

REFERENCES

Deng, J., Dong, W., Socher, R., Li, L., Kai Li,, & Li Fei-Fei,. (2009). ImageNet: A large-scale hierarchical image database. 2009 IEEE Conference on Computer Vision and Pattern Recognition, 248-255. https://doi.org/10.1109/cvpr.2009.5206848

Kingma, D. P., & Ba, J. (2015). Adam: A method for stochastic optimization. *International Conference on Learning Representations (ICLR 2015)*. https://arxiv.org/abs/1412.6980

Sandler, M., Howard, A., Zhu, M., Zhmoginov, A., & Chen, L.-C. (2018). MobileNetV2: Inverted residuals and linear bottlenecks. *Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition*, 4510–4520. https://doi.org/10.1109/CVPR.2018.00474

Sokolova, M., & Lapalme, G. (2009). A systematic analysis of performance measures for classification tasks. *Information Processing & Management, 45*(4), 427–437. https://doi.org/10.1016/j.ipm.2009.03.002

APPENDICES

Appendix A. Classification Report, Unseen Set (1,756 frames)

Class	Precision	Recall	F1-score	Support
A	0.997	0.997	0.997	336
B	1.000	0.997	0.999	359
C	1.000	1.000	1.000	378
D	1.000	1.000	1.000	366
E	0.997	1.000	0.998	317
Accuracy			**0.999**	**1,756**

Appendix B. Model Training and Evaluation Code (Google Colab)

```
# marker_training.py

# Converted from marker_training.ipynb.

# Section titles are comments; code below is preserved from the notebook.
```

```
# Note: lines beginning with ! are Google Colab/IPython shell commands and
are not valid in a standard Python interpreter.

# ================================================================

# 1. Load dataset

# ================================================================

from google.colab import drive

drive.mount('/content/drive')



!unzip -q -o /content/drive/MyDrive/dataset.zip -d /content

!unzip -q -o /content/drive/MyDrive/raw_test.zip -d /content



!ls /content/dataset/train /content/raw_test
```

```
# ================================================================

# 2. Build datasets

# ================================================================

import tensorflow as tf, numpy as np

from tensorflow import keras

from tensorflow.keras import layers
```

```
IMG, BS = (224, 224), 32



def load(path, shuffle=False):

    return keras.utils.image_dataset_from_directory(

        path, image_size=IMG, batch_size=BS, label_mode="int",

        shuffle=shuffle, seed=42)



train_ds = load("/content/dataset/train", shuffle=True)

val_ds   = load("/content/dataset/val")

test_ds  = load("/content/dataset/test")    # same conditions as training

dim_ds     = load("/content/raw_test")          # unseen: dim light + new
background



class_names = train_ds.class_names

print(class_names)

assert class_names == dim_ds.class_names == test_ds.class_names



AUTOTUNE = tf.data.AUTOTUNE

train_ds = train_ds.prefetch(AUTOTUNE)

val_ds = val_ds.prefetch(AUTOTUNE)
```

```
# ================================================================================

# 3. Model

# ================================================================================

aug = keras.Sequential([

    layers.RandomRotation(0.08),

    layers.RandomZoom(0.2),

    layers.RandomTranslation(0.1, 0.1),

    layers.RandomBrightness(0.5, value_range=(0, 255)),

    layers.RandomContrast(0.4),

], name="augmentation")

base = keras.applications.MobileNetV2(

    input_shape=IMG + (3,), include_top=False, weights="imagenet")

base.trainable = False

inputs = keras.Input(shape=IMG + (3,))

x = aug(inputs)

x = keras.applications.mobilenet_v2.preprocess_input(x)

x = base(x, training=False)

x = layers.GlobalAveragePooling2D()(x)

x = layers.Dropout(0.3)(x)
```

```
outputs = layers.Dense(len(class_names), activation="softmax")(x)

model = keras.Model(inputs, outputs)

model.compile(optimizer=keras.optimizers.Adam(1e-3),

              loss="sparse_categorical_crossentropy",

              metrics=["accuracy"])

model.summary()

# ============================================================================

# 4. Train the new head (base frozen)

# ============================================================================

h1 = model.fit(train_ds, validation_data=val_ds, epochs=10)

# ============================================================================

# 5. Fine-tune the top layers of MobileNetV2

# ============================================================================

base.trainable = True

for layer in base.layers[:-30]:

    layer.trainable = False

model.compile(optimizer=keras.optimizers.Adam(1e-5),

              loss="sparse_categorical_crossentropy",

              metrics=["accuracy"])
```

```
h2 = model.fit(train_ds, validation_data=val_ds, epochs=8)

# ============================================================================

# 6. Training curves (for the report)

# ============================================================================

import matplotlib.pyplot as plt

acc  = h1.history["accuracy"] + h2.history["accuracy"]

vacc = h1.history["val_accuracy"] + h2.history["val_accuracy"]

plt.plot(acc, label="train"); plt.plot(vacc, label="val")

plt.axvline(len(h1.history["accuracy"])   -   1,   color="gray",   ls="--",
label="fine-tune starts")

plt.xlabel("epoch");          plt.ylabel("accuracy");          plt.legend();
plt.title("Training curves")

plt.savefig("training_curves.png",      dpi=150,      bbox_inches="tight");
plt.show()

# ============================================================================

# 7. Evaluate on both test sets

# ============================================================================

from   sklearn.metrics   import   confusion_matrix,   classification_report,
ConfusionMatrixDisplay

def evaluate(ds, title, fname):

    y_true = np.concatenate([y.numpy() for _, y in ds])
```

```
    y_pred = model.predict(ds, verbose=0).argmax(1)

    print(f"=== {title} ===  accuracy: {(y_true == y_pred).mean():.4f}")

     print(classification_report(y_true, y_pred, target_names=class_names,
digits=3))

    ConfusionMatrixDisplay(confusion_matrix(y_true, y_pred),

                           display_labels=class_names).plot(cmap="Blues")

    plt.title(title)

    plt.savefig(fname, dpi=150, bbox_inches="tight"); plt.show()

    return y_true, y_pred

evaluate(test_ds, "Test set (same conditions)", "cm_test.png")

y_true_dim, y_pred_dim = evaluate(dim_ds, "Unseen set (dim light + new
background)", "cm_dim.png")

# ================================================================

# 8. Failure examples on the unseen set (for the report)

# ================================================================

images = np.concatenate([x.numpy() for x, _ in dim_ds]).astype("uint8")

wrong = np.where(y_true_dim != y_pred_dim)[0]

print(len(wrong), "misclassified of", len(y_true_dim))

if len(wrong):

       pick = np.random.RandomState(0).choice(wrong, min(9, len(wrong)),
replace=False)
```

```
    plt.figure(figsize=(10, 10))

    for k, idx in enumerate(pick):

        plt.subplot(3, 3, k + 1)

        plt.imshow(images[idx]); plt.axis("off")

                plt.title(f"true  {class_names[y_true_dim[idx]]}  /  pred
{class_names[y_pred_dim[idx]]}")

    plt.savefig("failures.png", dpi=150, bbox_inches="tight"); plt.show()

# ================================================================

# 9. Save the model

# ================================================================

model.save("marker_model.keras")

import json, shutil

json.dump(class_names, open("class_names.json", "w"))

for   f   in   ["marker_model.keras",   "class_names.json",   "cm_test.png",
"cm_dim.png",

          "training_curves.png", "failures.png"]:

    try: shutil.copy(f, "/content/drive/MyDrive/" + f)

    except FileNotFoundError: pass

print("saved to Drive")
```